LINUX ESSENTIALS
CHAPTER 1
COMMAND LINE BASICS.

Prepared by
Eng. Ibrahim Reyad
Email: ibrahimreyad1@gmail.com



Objective

- In this lab, you will perform the following tasks:
  - Explore Bash features
  - Use shell variables
  - Understand how to use globbing
  - Be able to make use of quoting

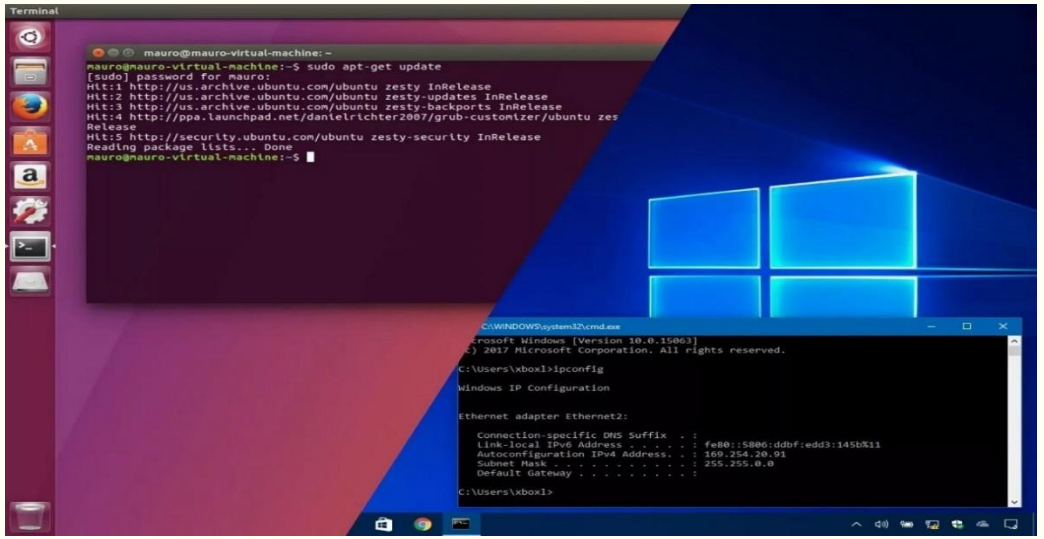
4.1 INTRODUCTION

4.1 Introduction

- **Graphical User Interface (GUI)**
- Most people familiar with using it to control your computer.

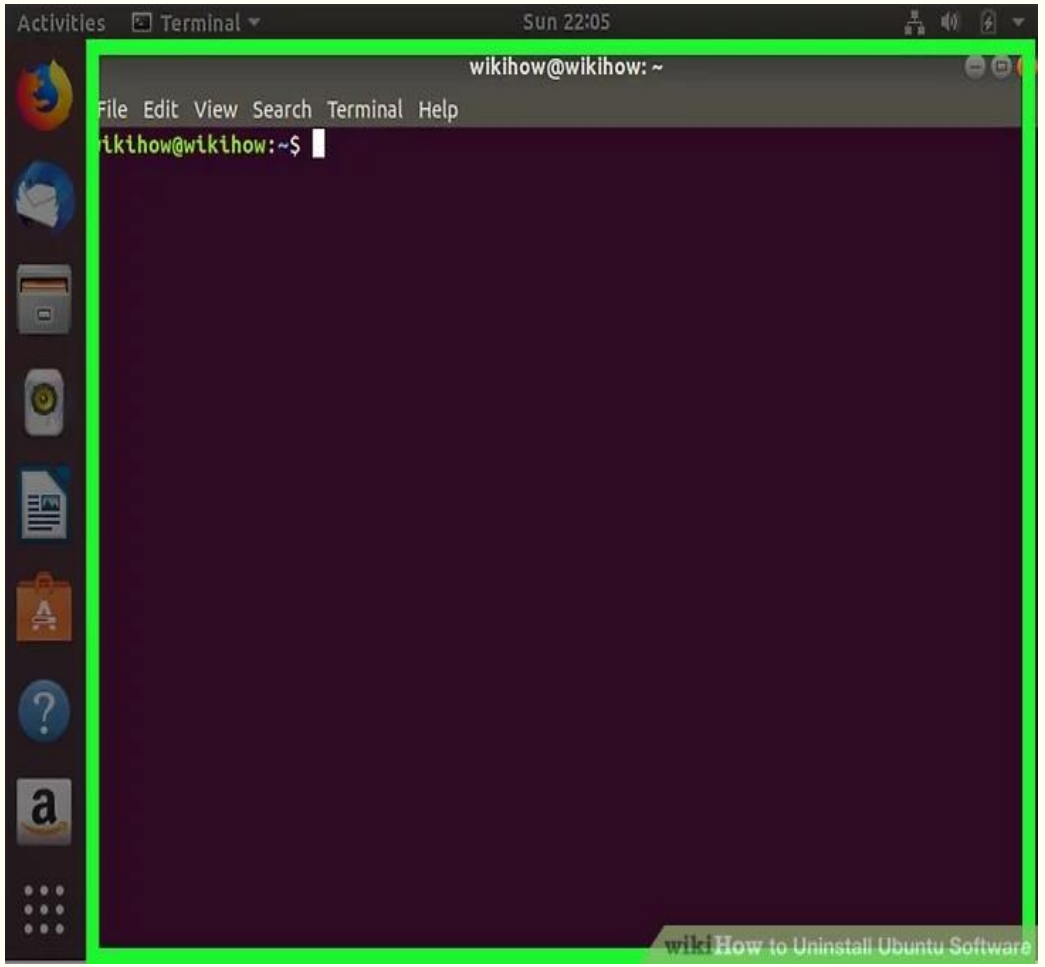
- **Command Line Interface (CLI)**
- relies on keyboard input. Everything you want the computer to do is relayed by typing commands rather than clicking on icons.





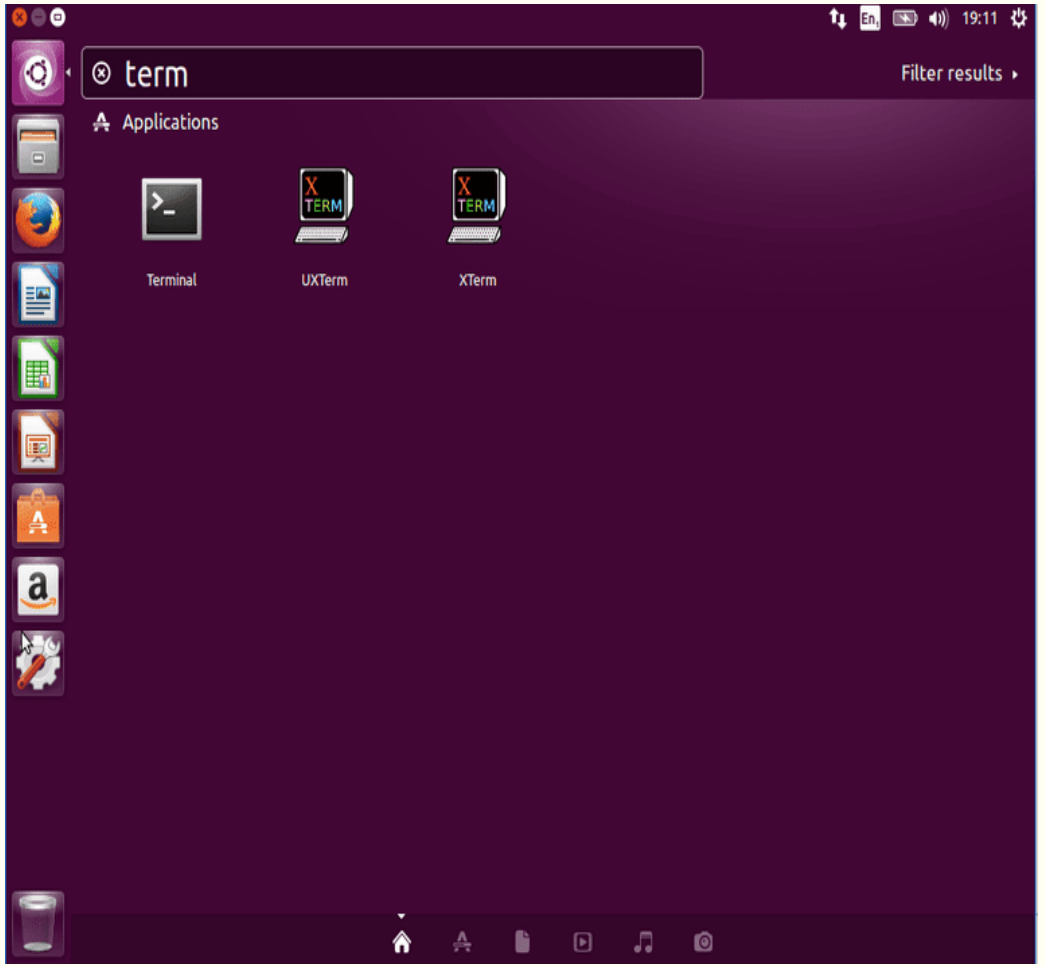
Command Line Interface (CLI)

- The CLI environment is provided by an application on the computer known as a **terminal**.
- The terminal accepts what the user types and passes to a **shell**.
- The shell interprets what the user has typed into instructions that can be executed by the operating system.



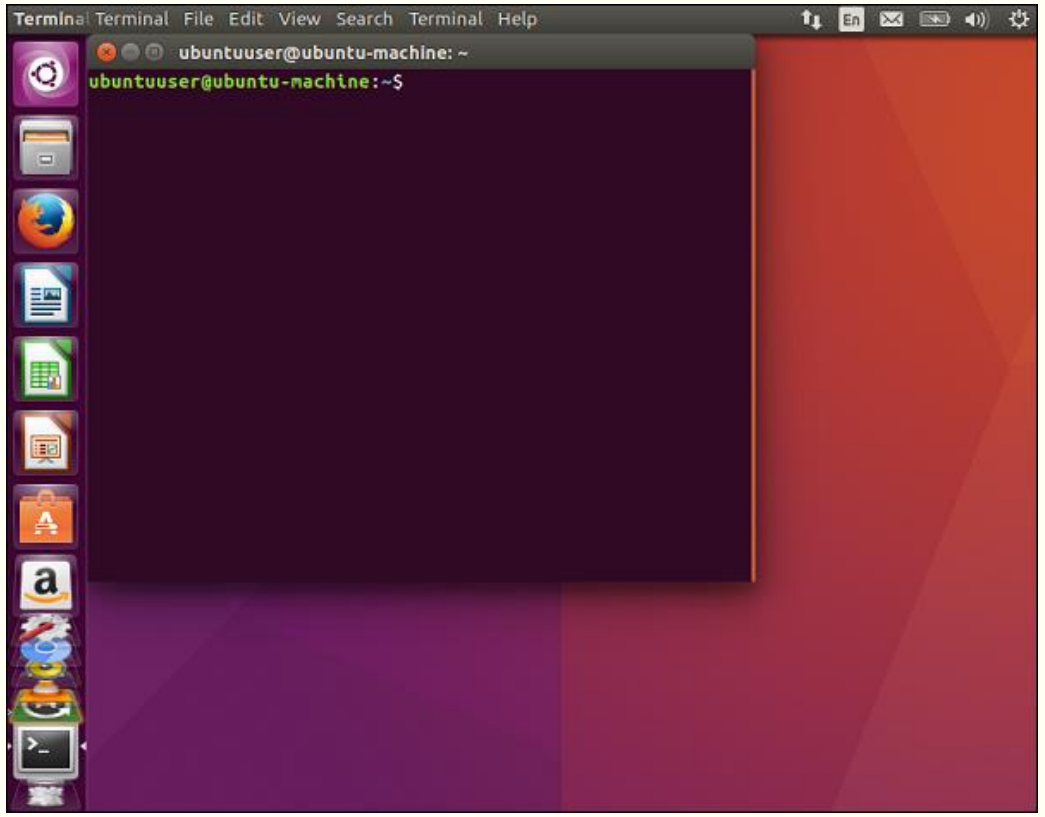
Accessing a Terminal

- There are many ways to access a terminal window.
  - You can open Terminal by directly pressing [ctrl+alt+T]
  - or you can search it up by clicking the "Dash" icon, typing in "terminal" in the search box, and opening the Terminal application.



4.2 Files and Directories

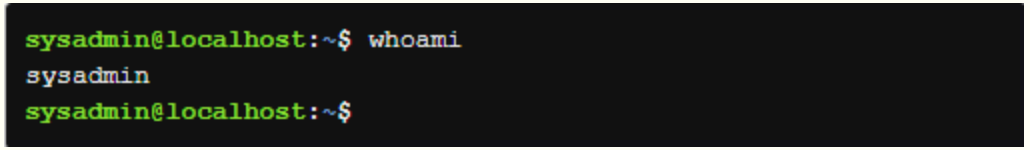
- A terminal window displays a **prompt**.
- It will typically contain information about the user and the system.
- It provides the name of the user that is logged in (sysadmin), the name of the system (localhost) and the current directory (~).
- The ~ symbol is used as shorthand for the user's home directory.




```
sysadmin@localhost:~$
```

4.2.1 Step 1

- The following command whoami
  - displays the user name of the current user
- Although in this case your username is displayed in the prompt, this command could be used to obtain this information in a situation when the prompt did not contain this information.



4.2.2 Step 2

- The following command uname
  - displays information about the current system.
- To be able to see the name of the kernel you are using, type the following command into the terminal
- Options for a command can be specified in several ways. Traditionally in UNIX, options were expressed by a hyphen followed by another character.

```
sysadmin@localhost:~$ uname
Linux
```

```
lab@ubuntu:~$ uname -n
ubuntu
lab@ubuntu:~$ uname --nodename
ubuntu
lab@ubuntu:~$
```

4.2.3 Step 3

- The following command pwd
  - display your current "location" or current "working" directory.
- pwd stands for "print working directory"

```
sysadmin@localhost:~$ pwd
/home/sysadmin
sysadmin@localhost:~$
```

SHELL VARIABLES

Shell variables are used to store data in Linux. This data is used by the shell itself as well as programs and users.

4.3.1 Step 1

- The following command echo
  - used to print text.
  - display the value of the PATH variable.

```
sysadmin@localhost:~$ echo Hello Student
Hello Student
sysadmin@localhost:~$
```

```
sysadmin@localhost:~$ echo $PATH
/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr
sysadmin@localhost:~$
```

- The following command which
  - determine if there is an executable file named date that is located in a directory listed in the PATH value.

```
sysadmin@localhost:~$ which date
/bin/date
sysadmin@localhost:~$
```

GLOBBING

The use of glob characters in Linux is similar to what many operating systems refer to as "wildcard" characters. Using glob characters, you match filenames using patterns.

Glob characters are a shell feature, not something that is particular to any specific command. As a result, you can use glob characters with any Linux command.

When glob characters are used, the shell will "expand" the entire pattern to match all files in the specified directory that matches the pattern.

For demonstration purposes, we will use the echo command to display this expansion process.

Globbing /wildcard (*)

- The following command echo
  - display all filenames in the current directory that matches the glob pattern *.
  - display all the files in the current directory that start with the letter D, and the letter P.
  - display all the files in your current directory that end in the letter s
- Notice that the asterisk can also appear multiple times or in the middle of several characters:

```
sysadmin@localhost:~$ echo *
Desktop Documents Downloads Music Pictures Public Templates Vide
sysadmin@localhost:~$
```

```
sysadmin@localhost:~$ echo D*
Desktop Documents Downloads
sysadmin@localhost:~$ echo P*
Pictures Public
sysadmin@localhost:~$
```

```
sysadmin@localhost:~$ echo *s
Documents Downloads Pictures Templates Videos
sysadmin@localhost:~$
```

Globbing /wildcard (?)

- The following command echo
  - display the filenames that are exactly six characters long:
  - display the file names that start with the letter D and are exactly nine characters long:
  - Wildcards or glob characters can be combined together to display file names that are at least six characters long and end in the letter s
- Notice that the asterisk can also appear multiple times or in the middle of several characters:

```
sysadmin@localhost:~$ echo ??????
Public Videos
sysadmin@localhost:~$
```

```
sysadmin@localhost:~$ echo D????????
Documents Downloads
sysadmin@localhost:~$
```

```
sysadmin@localhost:~$ echo ?????*s
Documents Downloads Pictures Templates Videos
sysadmin@localhost:~$
```

Glob Metacharacter/wildcard ([ ])

- This glob uses a pair of square brackets [ ] to specify which one character will be allowed. The allowed characters can be specified as a range, a list, or by what is known as a character class.
- The allowed characters can also be negated with an exclamation point !.
- In the first example, the first character of the file name can be either a D or a P. In the second example, the first character can be any character except a D or P.
- a range of characters will be specified.

```
sysadmin@localhost:~$ echo [DP]*
Desktop Documents Downloads Pictures Public
sysadmin@localhost:~$ echo [!DP]*
Music Templates Videos
sysadmin@localhost:~$
```

```
sysadmin@localhost:~$ echo [D-P]*
Desktop Documents Downloads Music Pictures Public
sysadmin@localhost:~$ echo [!D-P]*
Templates Videos
sysadmin@localhost:~$
```

Globbing /wildcard (quotes " ` ')

- There are three types of quotes used by the Bash shell:
  - single quotes (‘)
  - double quotes (")
  - back quotes (`).
- These quotes have special features in the Bash shell as we will describe.

Globbing /wildcard (back quotes `)

- to execute the date command within the line of the echo command:
  - Using back quotes(`).
  - place $(before the command and) after the command to accomplish command substitution.

```
sysadmin@localhost:~$ echo Today is `date`
Today is Tue Jan 19 15:48:57 UTC 2016
sysadmin@localhost:~$
```

```
sysadmin@localhost:~$ echo Today is $(date)
Today is Tue Jan 19 15:51:09 UTC 2016
sysadmin@localhost:~$
```

Globbing /wildcard (single quotes ')

- If you don't want the backquotes to be used to execute a command
  - place single quotes around them.
  - place a backslash character in front of each backquote character.

```
sysadmin@localhost:~$ echo This is the command ''date``'
This is the command `date`
sysadmin@localhost:~$
```

```
sysadmin@localhost:~$ echo This is the command \`date\`
This is the command `date`
sysadmin@localhost:~$
```

Globbing /wildcard (double quotes ")

- Double quote characters don't have any effect on backquote characters.

```
sysadmin@localhost:~$ echo This is the command "`date`"
This is the command Tue Jan 19 16:05:41 UTC 2016
sysadmin@localhost:~$
```

- Double quote characters will have an effect on wildcard characters, disabling their special meaning.

```
sysadmin@localhost:~$ echo D*
Desktop Documents Downloads
sysadmin@localhost:~$ echo "D*"
D*
sysadmin@localhost:~$
```

CONTROL STATEMENTS

Typically, you type a single command and you execute it when you press **Enter**. The Bash shell offers three different statements that can be used to separate multiple commands typed together.

But what about writing more than one command at the same line.

Control Statements(semicolon ;)

- using the semicolon
  - between multiple commands allows for them to be executed one right after another, sequentially from left to right.

```
sysadmin@localhost:~$ echo Hello; echo Linux; echo Student
Hello
Linux
Student
sysadmin@localhost:~$
```

  - put three commands together separated by semicolons, where the first command executes with a failure result.

```
sysadmin@localhost:~$ false; echo Not; echo Conditional
Not
Conditional
sysadmin@localhost:~$
```

- when commands are separated by the ; character, they are completely independent of each other

Control Statements(logical &&)

- Commands separated by && are conditionally executed. If the command on the left of the && is successful, then the command to the right of the && will also be executed. If the command to the left of the && fails, then the command to the right of the && is not executed.

```
sysadmin@localhost:~$ echo Start && echo Going && echo Gone
Start
Going
Gone
sysadmin@localhost:~$
```

```
sysadmin@localhost:~$ echo Success && false && echo Bye
Success
sysadmin@localhost:~$
```

Control Statements(logical ||)

- When commands are separated by ||, then only if the command to the left fails, does the command to the right of the || execute. If the command to the left of the || succeeds, then the command to the right of the || will not execute.

```
sysadmin@localhost:~$ false || echo Fail Or
Fail Or
sysadmin@localhost:~$ true || echo Nothing to see here
sysadmin@localhost:~$
```

SHELL HISTORY

The Bash shell maintains a history of the commands that you type. Previous commands can be easily accessed in this history in several ways.

shell history

- to recall a previous command is
  - use the **up arrow key**

  When you find the command that you want to execute, you can use the left arrow keys and right arrow keys to position the cursor for editing. Other useful keys for editing include the Home, End, Backspace and Delete keys.

  - use your command **history**
    - display the last five commands from your history:
    - To execute a command again, type the exclamation point and the history list number.

```
sysadmin@localhost:~$ echo Hi
Hi
sysadmin@localhost:~$ history
    1 date
    2 clear
    3 echo Hi
    4 history
sysadmin@localhost:~$
```

```
sysadmin@localhost:~$ history 5
  185  false || Fail Or
  186  false || echo Fail Or
  187  true || echo Nothing to see here
  188  history
  189  history 5
sysadmin@localhost:~$
```


Thank
you